

```

1  # Note 07: Logistic Regression and Sigmoid Function
2  # =====
3
4  # Import Libraries
5  import numpy as np
6  import matplotlib.pyplot as plt
7  import pandas as pd
8  from sklearn.model_selection import train_test_split
9  from sklearn.linear_model import LogisticRegression
10 from sklearn.metrics import accuracy_score, classification_report
11 from statsmodels.miscmodels.ordinal_model import OrderedModel
12
13 # 01. Plot Sigmoid Function
14 # =====
15
16 x = np.linspace(-10, 10, 1000)
17 y = 1 / (1 + np.exp(-x))
18 plt.figure()
19 plt.plot(x, y)
20
21 # Plot X = 0 Type Line
22 plt.axvline(x = 0, color = 'red', linestyle = '--')
23
24 # Plot Y = 0 Type Line
25 plt.axhline(y = 0.5, color = 'green', linestyle = '--')
26
27 plt.grid(True, alpha=0.3)
28 plt.title("Sigmoid Function")
29 plt.xlabel("X Values")
30 plt.ylabel("Y Values")
31 plt.show()
32
33 # 02. Logistic Regression - Binary Logistic Regression
34 # =====
35
36 # Header
37 print("\n=====")
38 print("01. Logistic Regression - BLR")
39 print("=====\n")
40
41 # Load Dataset
42 df = pd.read_csv("Sugar_Data.csv")
43
44 # Define X and Y Variables
45 x = df['Age']
46 y = df['AssignValue']
47
48 # Convert to NP array and Reshape
49 X = np.array(x).reshape(-1,1)
50 # -1 => Automatically Get the Count of Rows (Samples)
51 # 1 => One Column (One Feature)
52 Y = np.array(y)
53 # Length of X Array
54 print("Length of the X Array :", len(X))
55
56 # Train - Test Split
57 x_train, x_test, y_train, y_test = train_test_split(X, Y, test_size=0.7, random_state=
42)
58
59 # Print Data
60 print("\nX Original Array :", df['Age'].tolist())
61 print("Y Original Array :", Y.tolist())
62 print("\nX Train          :", (x_train.reshape(1, -1).flatten()).tolist()) # flatten:
Copy of 2D Array as a List
63 print("Y Train          :", (y_train.reshape(1, -1).flatten()).tolist()) # flatten:
Copy of 2D Array as a List
64 print("\nX Test          :", x_test.reshape(1, -1).flatten()).tolist()) # flatten:
Copy of 2D Array as a List
65 print("Y Test          :", y_test.reshape(1, -1).flatten()).tolist()) # flatten: Copy
of 2D Array as a List
66
67 # Logistic Regression

```

```

68 model = LogisticRegression(random_state=42)
69 model.fit(x_train, y_train)
70 predicted = model.predict(x_test)
71
72 # Print Results
73 print("\nThe predicted results are =", [int(x) for x in predicted])
74 print("The exact values for test =", [int(x) for x in y_test])
75
76 # Calculate Accuracy Score
77 accuracy = accuracy_score(y_test, predicted)
78 print("\nThe accuracy score is =", round((accuracy * 100), 2), "\b%")
79 print("The accuracy score is =", round((model.score(x_test, y_test) * 100), 2),
80       "\b%")
81
82 # Manual Calculation
83 acc = (np.sum(predicted == y_test) / len(y_test)) * 100
84 print("\nThe accuracy score is [Manual] = ", acc)
85
86 # Classification Report
87 print("\n=====")
88 print("Classification Report")
89 print("=====\n")
90
91 # Print Classification Report
92 print(classification_report(y_test, predicted))
93
94 # Predict a New Result
95 new_age = [[18]]
96 prediction = model.predict(new_age)
97 probability = model.predict_proba(new_age)
98
99 print("Predcition of Sugar For Age 18 :", prediction[0])
100 print("Probability of Sugar For Age 18 :", round((probability[0, 0] * 100), 2), "\b%")
101 # Proba = [[P(Class 0), P(Class 1)]]
102
103 # 03. Train - Test Split
104 # =====
105
106 # Header
107 print("\n=====")
108 print("02. Train - Test Split")
109 print("=====\n")
110
111 # Load Dataset
112 df = pd.read_csv("trail.csv")
113 print("Column Names: ", df.columns.tolist())
114
115 # Define X and Y Variables
116 x = df.age
117 y = df.AssignValue
118
119 # Print the Type of X and Y
120 print("\nType of X :", type(x).__name__)
121 print("Type of Y :", type(y).__name__)
122
123 # Convert to Numpy Arrays
124 X = np.array(x)
125 Y = np.array(y)
126
127 # Print Types
128 print("\nType of X :", type(X).__name__)
129 print("Type of Y :", type(Y).__name__)
130
131 # NP Array Shape
132 a = np.array([1, 2, 3])
133 print("\nType of a :", type(a).__name__)
134 print("Shape of a :", df.shape)
135
136 # Train - Test Split
137 x_train, x_test, y_train, y_test = train_test_split(X, Y, shuffle = False, test_size=
0.8, random_state=42)
138 # Shuffle: True [Best One]

```

```

138 print("\nLength of X Array :", len(X))
139 print("Length of X Train :", len(x_train))
140 print("Length of X Test  :", len(x_test))
141 print("Original X Array  :", X.tolist())
142 print("X Train Array     :", x_train.tolist())
143
144 # 04. Logistic Regression - Multinomial Regression
145 # =====
146
147 # Header
148 print("\n=====")
149 print("03. Logistic Regression - MLR")
150 print("=====\\n")
151
152 # Dataset
153 data = {
154     "age": [18, 20, 22, 25, 30, 35, 40, 45, 50, 55, 60, 65],
155     "income": [20, 22, 25, 30, 35, 40, 45, 50, 55, 60, 65, 70],
156     "transport": [0, 0, 0, 1, 1, 1, 1, 2, 2, 2, 2, 2]
157 }
158
159 # Data Frame
160 df = pd.DataFrame(data)
161
162 # Features
163 X = df[["age", "income"]]
164
165 # Target
166 y = df["transport"]
167
168 # Train-test split
169 X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.5, random_state=42)
170
171 # Multinomial Logistic Regression
172 # [multi_class="multinomial": Depreciated / Detect Automatically]
173 model = LogisticRegression(max_iter=1000)
174
175 # Train
176 model.fit(X_train, y_train)
177
178 # Prediction
179 y_pred = model.predict(X_test)
180
181 # Evaluation
182 print("Predictions:", y_pred)
183 print("Actual:", y_test.values)
184 print("Accuracy:", accuracy_score(y_test, y_pred))
185 print("\\n", classification_report(y_test, y_pred))
186
187 # Predict a new person
188 new_person = pd.DataFrame([[32, 42]], columns = ["age", "income"])
189 prediction = model.predict(new_person)
190 probability = model.predict_proba(new_person)
191 print("Predicted Class:", prediction[0])
192 print("Probabilities:", probability[0, 1])
193
194 # 05. Logistic Regression - Ordinary Logistic Regression
195 # =====
196
197 # Header
198 print("\n=====")
199 print("03. Logistic Regression - OLR")
200 print("=====\\n")
201
202 # Dataset
203 data = {
204     "hours": [1, 2, 2, 3, 3, 4, 4, 5, 5, 6, 6, 7],
205     "score": [30, 35, 40, 45, 50, 55, 60, 65, 70, 75, 80, 90],
206     "level": ["Low", "Low", "Low", "Medium", "Medium", "Medium", "Medium", "High",
207             "High", "High", "High", "High"]
208 }

```

```

208
209 # Data Frame
210 df = pd.DataFrame(data)
211
212 # Convert target into ordered categories
213 df["level"] = pd.Categorical(
214     df["level"],
215     categories=["Low", "Medium", "High"],
216     ordered=True)
217
218 # Features
219 X = df[["hours", "score"]]
220
221 # Target
222 y = df["level"]
223
224 # Create ordinal logistic regression model
225 model = OrderedModel(y, X, distr="logit")
226 # distr="probit" for normal distribution
227
228 # Train model
229 result = model.fit(method="bfgs")
230 # Finding the coefficients that maximize the likelihood
231
232 # Display results
233 print(result.summary())
234
235 # Predict
236 new_data = pd.DataFrame({
237     "hours": [5],
238     "score": [68]
239 })
240
241 prediction = result.model.predict(
242     result.params,
243     exog=new_data # Predict using new_data as the input features
244 )
245
246 print("\nPrediction: ", prediction)
247

```